



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U03_A02 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 16 de Noviembre, 2025

ÍNDICE

INTRODUCCIÓN 3

ENUNCIADO..... 4

PROBAD TODO EL CÓDIGO DEL APARTADO 2 4

PROBAD LOS SCRIPTS DEL APARTADO 2 8

CASO PRÁCTICO 2 10

OBJETOS MANEJADORES DE EVENTOS 10

PONTE A PRUEBA 2..... 11

DELEGACIÓN DE EVENTOS 11

CLAVES DE RESOLUCIÓN: 12

DESARROLLO DE LA PRACTICA..... 13

DESCRIPCIÓN..... 13

1. **CASO PRÁCTICO 2:** CREACIÓN DE OBJETOS MANEJADORES DE EVENTOS PARA MOSTRAR RESPUESTAS DE UN ARRAY DE CHISTES..... 13

2. **PONTE A PRUEBA 2:** IMPLEMENTACIÓN DE LA DELEGACIÓN DE EVENTOS DE UNA REJILLA DE CELDAS PARA ACTIVAR O DESACTIVAR SU ESTADO..... 13

CONTENIDO DEL ARCHIVO..... 13

CÓDIGO DEL APARTADO 2 DE TEORÍA 13

CASO PRÁCTICO 2 – OBJETOS MANEJADORES DE ARCHIVOS 20

PONTE A PRUEBA 2 – DELEGACIÓN DE EVENTOS 22

CÓMO VISUALIZARLO 23

REQUISITOS: 23

PASOS: 23

OBJETIVOS DE LA PRÁCTICA..... 23

- **PROBAR Y COMPRENDER EL CÓDIGO TEÓRICA DEL APARTADO 2..... 23**
- **CREAR MANEJADORES DE EVENTOS ASOCIADOS A ELEMENTOS DINÁMICOS. 23**
- **IMPLEMENTAR DELEGACIÓN DE EVENTOS PARA OPTIMIZAR LA GESTIÓN DE INTERACCIONES. 23**
- **DOCUMENTAR CORRECTAMENTE, COMO SIEMPRE, CADA EJERCICIO CON NOMBRE, FECHA Y CAPTURAS DE PANTALLA. 23**
- **PRESENTAR EL PROYECTO EN FORMATO PDF CON LOS ARCHIVOS FUENTE Y PANTALLAZOS..... 23**

CONCLUSIÓN 24

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 2” deberemos incluir en uno o varios (yo personalmente recomiendo siempre varias fuentes) todo el código que aparece en el apartado 2 de la teoría de JavaScript, así como su ejecución y autoría.

En el ejercicio “Probad los Scripts del Apartado 2” deberemos ejecutar los scripts adjuntos y explicaremos, asimismo, su funcionamiento incluyendo pruebas de autoría.

En el “Caso práctico 2” vamos a crear un programa en JavaScript que muestre para cada chiste un elemento <div> que contenga el enunciado del propio chiste y la respuesta. Al pulsar el botón, se mostrará un “alert” con la respuesta correspondiente.

En el “Ponte a prueba 2” crearemos el código JS necesario para implementar la funcionalidad indicada de la manera más eficiente posible.

ENUNCIADO

Probad todo el código del Apartado 2

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de ese apartado, así como su ejecución y prueba de autoría.

Ejemplo de implementación:

Captura de pantalla – DWEC_U03_A02_TEO_1.html

```
<> DWEC_U03_A02_TEO_1.html X
<> DWEC_U03_A02_TEO_1.html > html > body > script
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 15/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>2.1. Eventos: tipos</title>
10     </head>
11     <body>
12         <button id="boton" onclick="saludar()">Pulsa el botón</button>
13
14         <script>
15             function saludar() {
16                 alert("Hola Mundo!");
17             }
18
19             document.getElementById("boton").onclick = saludar;
20         </script>
21
22         <!-- Prueba de autoría -->
23         <br><br><p>Autor: <strong>Mario Segura Abad</strong></p>
24         <p>Fecha de realización: 15/11/2025</p>
25     </body>
26 </html>
```

Captura de pantalla – DWEU03_A02_TEO_2.html

```
<> DWEU03_A02_TEO_2.html X
<> DWEU03_A02_TEO_2.html > html > body > p
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 15/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>2.1. Eventos: tipos</title>
10     </head>
11     <body>
12         <button id="boton">Pulsa el botón</button>
13
14         <script>
15             function saludar1(): void
16             {
17                 alert("Hola Mundo!");
18             }
19
20             function saludar2() {
21                 alert("Hola otra vez!");
22             }
23
24             let boton = document.getElementById("boton");
25             boton.addEventListener("click", saludar1);
26
27             // Se puede añadir un segundo manejador de eventos, que se ejecutará tras el anterior
28             boton.addEventListener("click", saludar2);
29         </script>
```

Captura de pantalla – DWEU03_A02_TEO_3.html

```
<> DWEU03_A02_TEO_3.html X
<> DWEU03_A02_TEO_3.html > html > body > p
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 15/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>2.1. Eventos: tipos</title>
10     </head>
11     <body>
12         <button id="boton">Pulsa el botón</button>
13
14         <script>
15             function saludar1() {
16                 alert("Hola Mundo!");
17             }
18
19             let boton = document.getElementById("boton");
20
21             // Añadir manejador de eventos
22             boton.addEventListener("click", saludar1);
23
24             // Código...
25
26             // Eliminar manejador de eventos
27             // Hay que indicar de nuevo el evento y la función asociada
28             boton.removeEventListener("click", saludar1);
29         </script>
30
```

Captura de pantalla – DWEC_U03_A02_TEO_4.html

```
<> DWEC_U03_A02_TEO_4.html X
<> DWEC_U03_A02_TEO_4.html > html > body > p
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 15/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>2.1. Eventos: tipos</title>
10     </head>
11     <body>
12         <button id="boton">Pulsa el botón</button>
13
14         <script>
15             function saludar(evento) {
16                 console.log(`El evento se ha producido sobre el objeto ${this}`);
17                 console.log(`Evento de tipo ${evento.type} sobre el elemento ${evento.currentTarget}.`);
18             }
19
20             let boton = document.getElementById("boton");
21             boton.addEventListener("click", saludar);
22         </script>
23
24         <!-- Prueba de Autoría -->
25         <br><br><p>Autor: <strong>Mario Segura Abad</strong></p>
26         <p>Fecha de realización: 15/11/2025</p>
27     </body>
28 </html>
```

Captura de pantalla – DWEC_U03_A02_TEO_5.html

```
<> DWEC_U03_A02_TEO_5.html X
<> DWEC_U03_A02_TEO_5.html
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>2.2. Propagación de eventos</title>
10     </head>
11     <body>
12         <div>
13             <p>Párrafo</p><br>
14         </div>
15
16         <div onclick="alert('Click!')">
17             <p>Párrafo 1</p>
18             <p>Párrafo 2</p>
19             <p>Párrafo 3</p>
20         </div>
21
22         <!-- Prueba de autoría -->
23         <br><p>Autor: <strong>Mario Segura Abad</strong></p>
24         <p>Fecha de realización: 15/11/2025</p>
25     </body>
26 </html>
```

Captura de pantalla – DWECE_U03_A02_TEO_6.html

```
<> DWECE_U03_A02_TEO_6.html X
<> DWECE_U03_A02_TEO_6.html > html > body > br
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>2.4. Comportamiento por defecto</title>
10     </head>
11     <body>
12         <a href="https://google.es" onclick="accion1(event)">Ir a Google</a><br>
13         <a href="https://google.es" onclick="accion2(event)">Ir a Google</a>
14
15         <script>
16             function accion1(evento) {
17                 console.log("Se ejecuta el código y se va a Google");
18             }
19
20             function accion2(evento) {
21                 evento.preventDefault();
22                 console.log("Se ejecuta el código y NO se va a Google");
23             }
24         </script>
25
26         <!-- Prueba de Autoría -->
27         <br><br><br><p>Autor: <strong>Mario Segura Abad</strong></p>
28         <p>Fecha de realización: 15/11/2025</p>
29     </body>
30 </html>
```

Captura de pantalla – DWECE_U03_A02_TEO_7.js

```
JS DWECE_U03_A02_TEO_7.js X
JS DWECE_U03_A02_TEO_7.js > [?] editarPersona
1  // Autor: Mario Segura Abad
2  // Fecha: 16/11/2025
3
4  let personas = [
5      {
6          nombre: "Ana",
7          apellido: "López"
8      },
9      {
10         nombre: "Inés",
11         apellido: "García"
12     }
13 ];
14
15 let editarPersona = {
16     handleEvent: function(evento) {
17         alert(`Editando datos de ${this.datosPersona.nombre} ${this.datosPersona.apellido}`);
18     }
19 }
20
21 let borrarPersona = {
22     handleEvent: function(evento) {
23         alert(`Borrando datos de ${this.datosPersona.nombre} ${this.datosPersona.apellido}`);
24     }
25 }
```

Probad los Scripts del Apartado 2

Ejecutad los scripts adjuntos y explicad su funcionamiento. Recordad incluir pruebas de autoría.

Ejemplo de implementación:

Captura de pantalla – DWEC_U03_A02_01.html

```
<> DWEC_U03_A02_01.html ×
Scripts Apartado 2 > <> DWEC_U03_A02_01.html > p
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/11/2025 -->
3
4  <div id="contenedor">
5      <!-- Tres párrafos dentro del div -->
6      <p>Párrafo 1</p>
7      <p>Párrafo 2</p>
8      <p>Párrafo 3</p>
9  </div>
10
11 <script>
12     // Esta función será llamada cada vez que se haga click dentro del contenedor.
13     // "evento" contiene toda la información del click.
14     function procesarEvento(evento) {
15
16         // evento.target -> el elemento exacto donde el usuario hizo click.
17         // Si clicas sobre el texto "Párrafo 2", evento.target será <p>
18         alert(`El evento se ha producido sobre el objeto ${evento.target.tagName}`);
19
20         // evento.currentTarget -> el elemento que tiene asociado el listener.
21         // Aquí siempre será <div id="contenedor"> porque ahí pusimos addEventListener.
22         alert(`El manejador de eventos es el elemento ${evento.currentTarget.tagName}`);
23     }
24
25     // Obtenemos el DIV que contiene los párrafos
26     let boton = document.getElementById("contenedor");
27
28     // Añadimos un único listener al contenedor.
29     // Gracias a la delegación de eventos, también detecta clicks en los <p>.
30     boton.addEventListener("click", procesarEvento);
```


Captura de pantalla – DWEC_U03_A02_02.html

```

<> DWEC_U03_A02_02.html X
Scripts Apartado 2 > <> DWEC_U03_A02_02.html > br
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/11/2025 -->
3
4  <style>
5      .acordeon {
6          <element class="acordeon-titulo">
7              Selector Specificity: (0, 1, 0)
8      }
9      .acordeon-titulo {
10         cursor: pointer;
11         margin-bottom: 0.5em;
12     }
13     .acordeon-contenido {
14         max-height: 0;
15         overflow: hidden;
16         transition: max-height 0.5s;
17     }
18     .acordeon-contenido-visible {
19         border-top: 1px solid black;
20         max-height: 5em;
21         transition: max-height 0.5s;
22         overflow: scroll;
23     }
24 </style>
25
26 <div class="acordeon" data-acordeon>
27     <div class="acordeon-titulo" data-acordeon-titulo>
28         Título 1
29

```

Captura de pantalla – DWEC_U03_A02_03.html

```

<> DWEC_U03_A02_03.html X
Scripts Apartado 2 > <> DWEC_U03_A02_03.html > ...
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/11/2025 -->
3
4  <div id="personas">
5      <!-- Aquí se mostrará el listado de personas junto con sus botones de edición y borrado -->
6  </div>
7
8  <script>
9      // Datos
10     let personas = [
11         {
12             nombre: "Ana",
13             apellido: "López"
14         },
15         {
16             nombre: "Inés",
17             apellido: "García"
18         }
19     ];
20
21     // Prototipos para los manejadores de eventos
22     // Definen el método 'handleEvent'
23     let editarPersona = {
24         handleEvent: function(evento) {
25             // Lógica para la tarea de edición
26             alert(`Editando datos de ${this.datosPersona.nombre} ${this.datosPersona.apellido}`);
27         }
28     };

```

Caso práctico 2

Objetos manejadores de eventos

Crea un programa en JavaScript que muestre para cada chiste un elemento **<div>** que contenga el enunciado y un botón con el texto **Ver respuesta**. Al pulsar en el botón se debe mostrar mediante un **alert**, la respuesta correspondiente. Adjunto dispones del archivo **DWEC_U03_A02_CP_E.js** con un *array* de objetos “chiste”.

Ejemplo de implementación:

Captura de pantalla – DWEC_U03_A02_CP_E.js

```
JS DWEC_U03_A02_CP_E.js ×
CasoPractico > JS DWEC_U03_A02_CP_E.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 17/11/2025
3  ⚡
4  let chistes = [
5      {
6          enunciado: "¿Qué le dice un bit a otro?",
7          respuesta: "Nos vemos en el bus."
8      },
9      {
10         enunciado: "¿Qué es un terapeuta?",
11         respuesta: "1024 GigaPeutas."
12     },
13     {
14         enunciado: "¿Cuántos programadores hacen falta para cambiar una bombilla?",
15         respuesta: "Ninguno, porque es un problema de hardware."
16     }
17 ]
18
```

Captura de pantalla – CasoPractico2.html

```
<> CasoPractico2.html X
CasoPractico > <> CasoPractico2.html > html > body > br
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/11/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>Caso Práctico 2</title>
10     </head>
11     <body>
12         <h2>Chistes</h2><br>
13         <div id="contenedor"></div>
14
15         <!-- Archivo que contiene el aarray "chistes" -->
16         <script src="DWECE_U03_A02_CP_E.js"></script>
17
18         <script>
19             // Objeto manejador de eventos
20             const verRespuesta = {
21                 respuesta: "",
22                 handleEvent(evento) {
23                     alert(this.respuesta);
24                 }
25             };
26
27             // Contenedor donde se mostrarán los chistes
28             const contenedor = document.getElementById("contenedor");
29
```

Ponte a prueba 2

Delegación de eventos

Se dispone de una rejilla formada por celdas que se quieren activar o desactivar pulsando sobre ellas. La activación o desactivación de una celda se traducirá en la presencia o ausencia de la clase **activa**.

Crea el código JavaScript necesario para implementar la funcionalidad indicada de la manera **más eficiente posible**.

Adjunto dispones del archivo **DWECE_U03_A02_PP_E.htm**, con el código HTML de partida.

Ejemplo de implementación:

```
<> DWEC_U03_A02_PP_E.html x
PonteAPrueba > <> DWEC_U03_A02_PP_E.html > html > body > br
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/11/2025 -->
3
4  <!DOCTYPE html>
5  <html>
6    <head>
7      <meta charset="UTF-8">
8      <title>Unidad 3 – Ponte a Prueba</title>
9      <style>
10         #rejilla {
11             width: 50vw;
12             height: 50vh;
13             margin: auto;
14             display: grid;
15             grid-template-columns: 1fr 1fr 1fr 1fr;
16             gap: 1em;
17             justify-content: stretch;
18         }
19
20         .celda {
21             cursor: pointer;
22             padding: 1em;
23             background-color: #c2d9ed;
24             display: flex;
25             align-items: center;
26             justify-content: center;
27         }
28
29         .activa {
30             background-color: #6ea8db;
```

Claves de resolución:

- Debes utilizar la característica de delegación de eventos del DOM: dado que todas las celdas están dentro de una capa contenedora, debes agregar al manejador a dicha capa y no a cada celda.
- Puesto que el manejador va a estar definido en la capa contenedora, también detectará el evento si se pulsa sobre ella. Tienes que **impedir** que si se pulsa directamente sobre dicha capa (pulsando en los **bordes** de las celdas, por ejemplo) se añada la clase **activa** al contenedor.

DESARROLLO DE LA PRACTICA

Práctica DWEK – Apartado 2 Tema 3

Descripción

Este apartado se centra en el uso de **eventos en JavaScript** y cómo gestionarlos de manera eficiente.

El objetivo principal ha sido probar todo el código teórico, ejecutar los scripts adjuntos y desarrollar dos ejercicios prácticos:

1. **Caso práctico 2:** creación de objetos manejadores de eventos para mostrar respuestas de un array de chistes.
2. **Ponte a prueba 2:** implementación de la delegación de eventos de una rejilla de celdas para activar o desactivar su estado.

Contenido del archivo

Código del Apartado 2 de teoría

Se ha recopilado y probado todo el código del apartado 2 de la teoría de JavaScript relacionado con:

- Definición de manejador de eventos (`addEventListener`).
- Uso de funciones anónimas y funciones flecha como callbacks.
- Delegación de eventos en elementos contenedores.
- Métodos del DOM para manipular clases (`classList.add`, `classList.remove`, `classList.toggle`).

Cada archivo incluye comentarios con el nombre del autor y la fecha de realización. Se han capturado los resultados por consola y pantalla como prueba de ejecución.

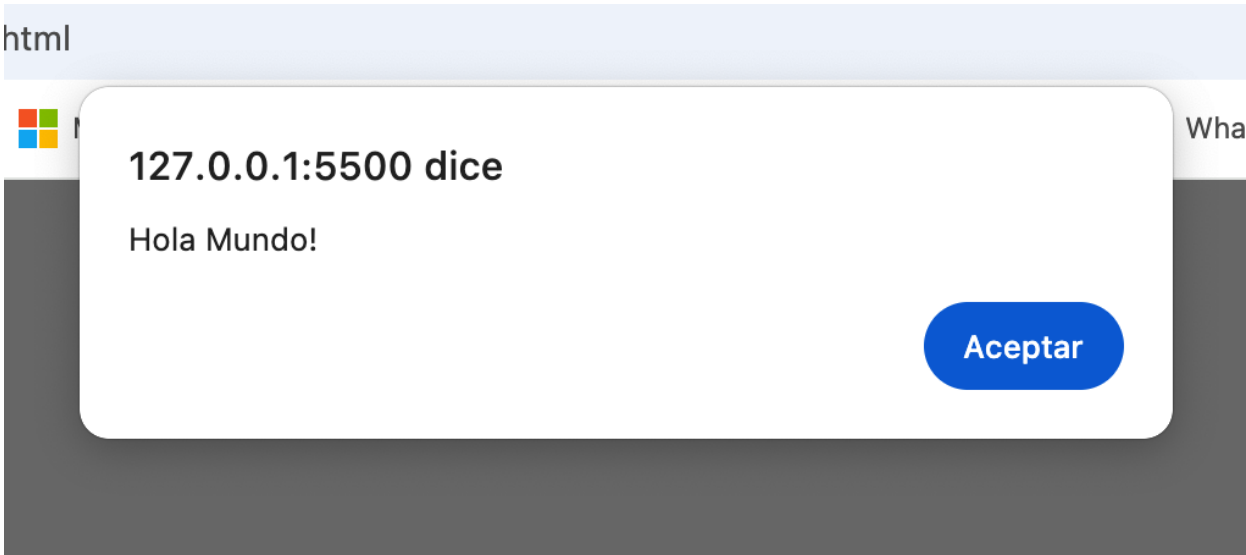
Ejemplo de ejecución:

Salida por consola – DWEK_U03_A02_TEO_1.html

Pulsa el botón

Autor: Mario Segura Abad

Fecha de realización: 15/11/2025

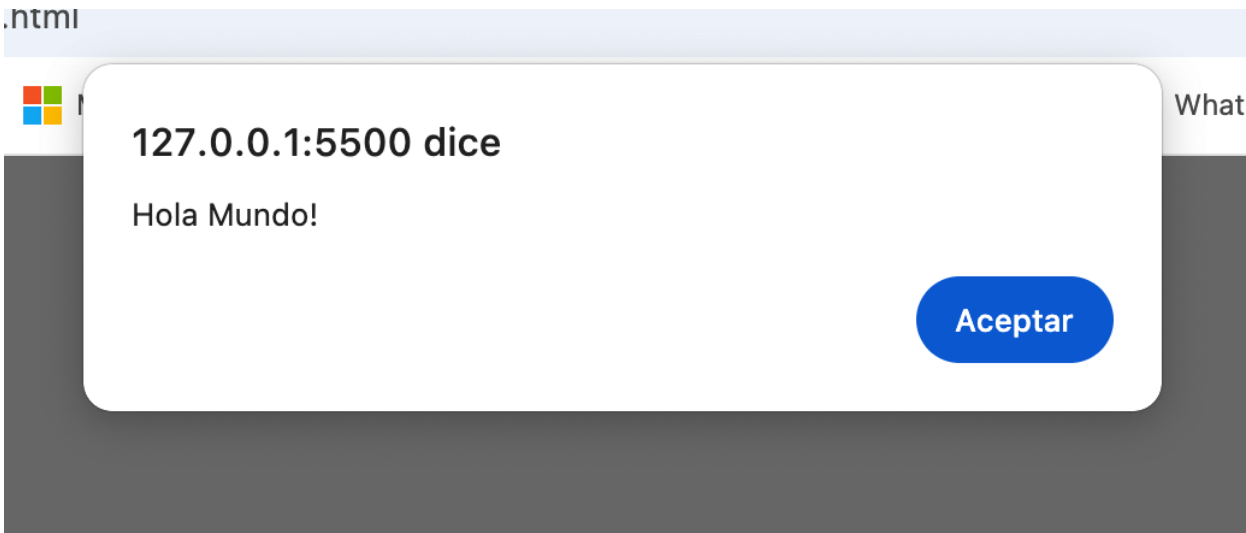


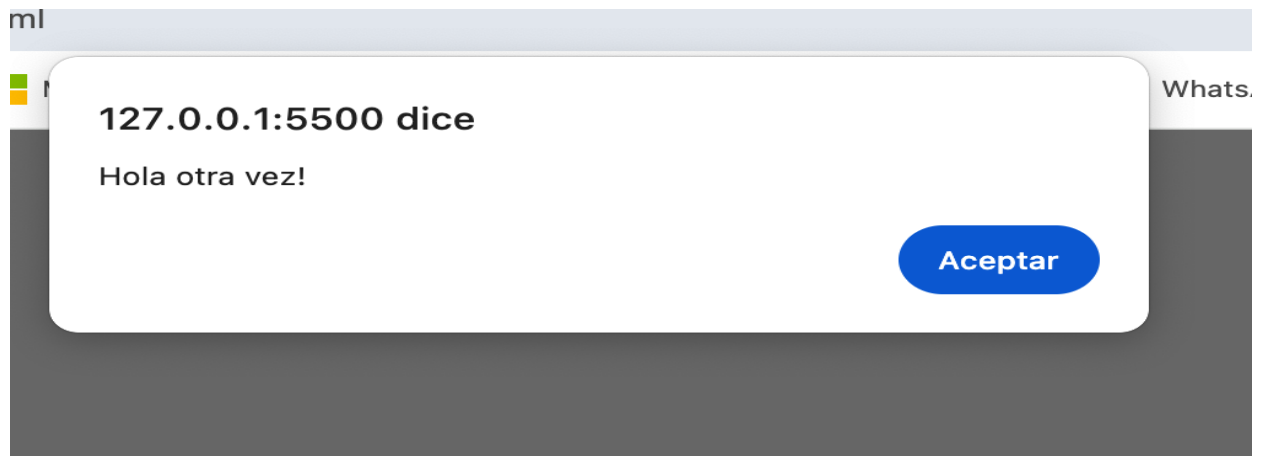
Salida por consola – DWEC_U03_A02_TEO_2.html

Pulsa el botón

Autor: **Mario Segura Abad**

Fecha de realización: 15/11/2025





Salida por consola – DWEK_U03_A02_TEO_3.html

Pulsa el botón

Autor: **Mario Segura Abad**

Fecha de realización: 15/11/2025

Salida por consola – DWEK_U03_A02_TEO_4.html hace exactamente lo mismo que el TEO_3.html, o sea, no hace absolutamente nada cuando pulsas el botón, pues paso directamente a la TEO_5.html que sí cambia.

Salida por consola – DWEC_U03_A02_TEO_5.html

Párrafo

Párrafo 1

Párrafo 2

Párrafo 3

Autor: **Mario Segura Abad**

Fecha de realización: 15/11/2025

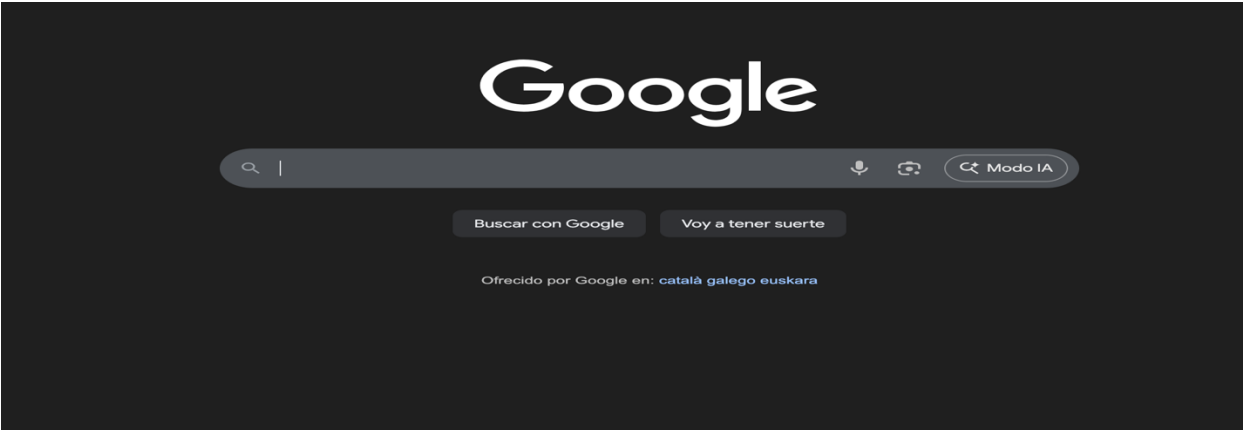
Salida por consola – DWEC_U03_A02_TEO_6.html

[Ir a Google](#)

[Ir a Google](#)

Autor: **Mario Segura Abad**

Fecha de realización: 15/11/2025



Salida por consola – DWEC_U03_A02_TEO_7.js

```
,  
  
// Prueba de Autoría  
console.log("Autor: Mario Segura Abad");  
console.log("Fecha de realización: 16/11/2025");  
_____  
Autor: Mario Segura Abad  
_____  
Fecha de realización: 16/11/2025  
_____  
⏪ undefined
```

Salida por consola – DWEC_U03_A02_01.html

Párrafo 1

Párrafo 2

Párrafo 3

Autor: **Mario Segura Abad**

Fecha de realización: 16/11/2025

Salida por consola – DWEC_U03_A02_02.html

Título 1

Título 2

Autor: **Mario Segura Abad**

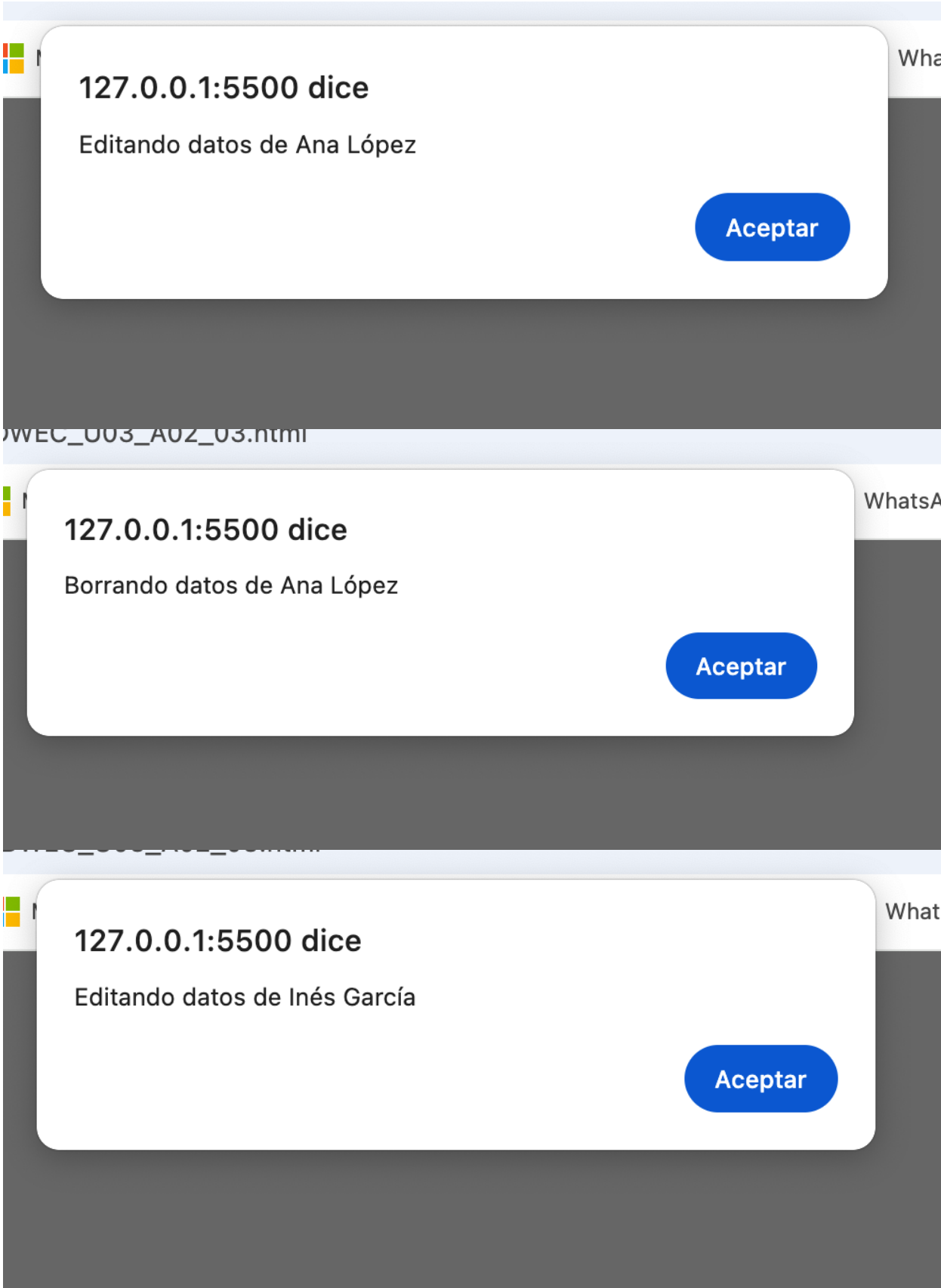
Fecha de realización: 16/11/2025

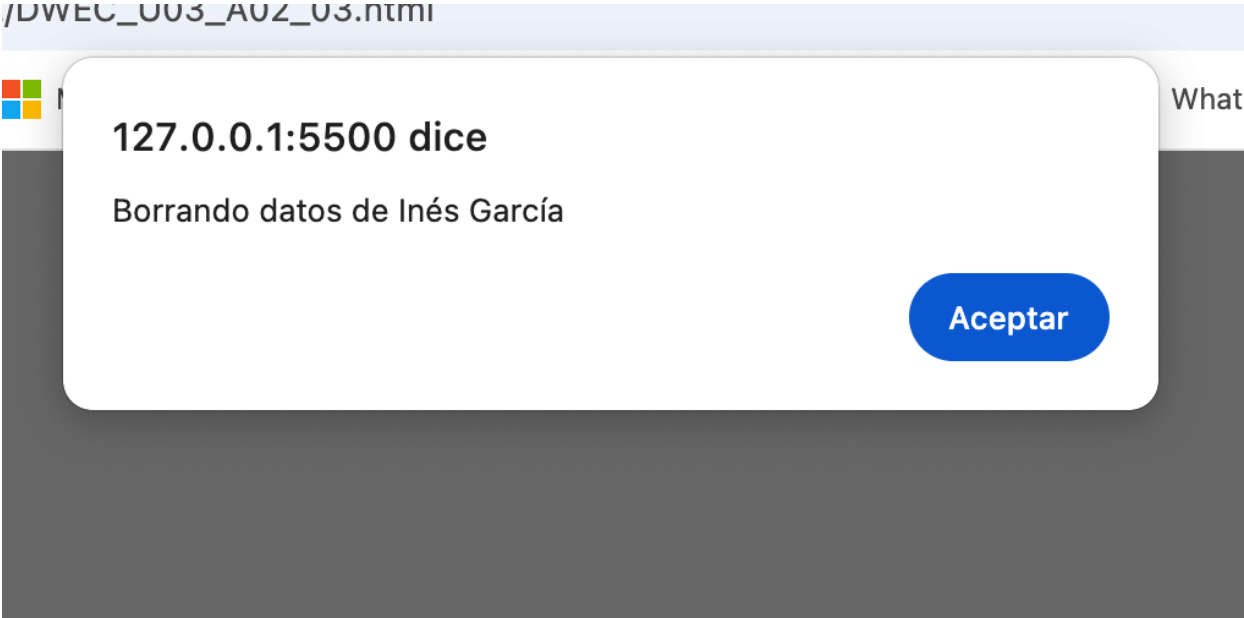
Salida por consola – DWEC_U03_A02_03.html

Ana López	Editar	Borrar
Inés García	Editar	Borrar

Autor: **Mario Segura Abad**

Fecha de realización: 16/11/2025





Caso Práctico 2 – Objetos manejadores de archivos

Se ha creado un programa en JavaScript que:

- Recorre un array de objetos “chiste” (con enunciado y respuesta).
- Genera dinámicamente un <div> para cada chiste con su enunciado y un botón **Ver respuesta**.
- Al pulsar el botón, se muestra la respuesta correspondiente mediante un alert.

Este ejercicio ha reforzado el uso de **eventos asociados a botones** y la creación dinámica de elementos en el DOM.
Ejemplo de ejecución:

Chistes

¿Qué le dice un bit a otro?

Ver respuesta

¿Qué es un terapeuta?

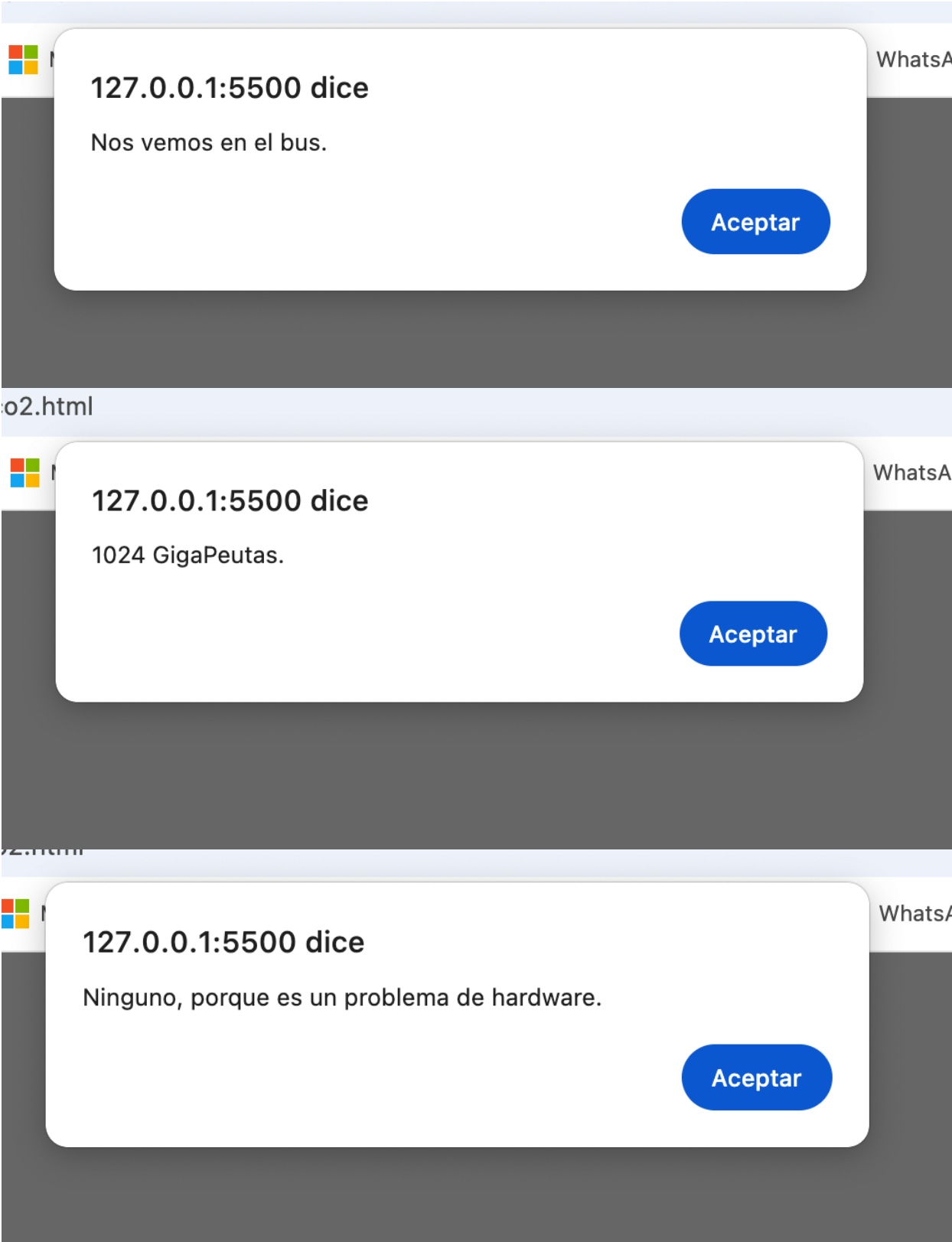
Ver respuesta

¿Cuántos programadores hacen falta para cambiar una bombilla?

Ver respuesta

Autor: **Mario Segura Abad**

Fecha de realización: 16/11/2025



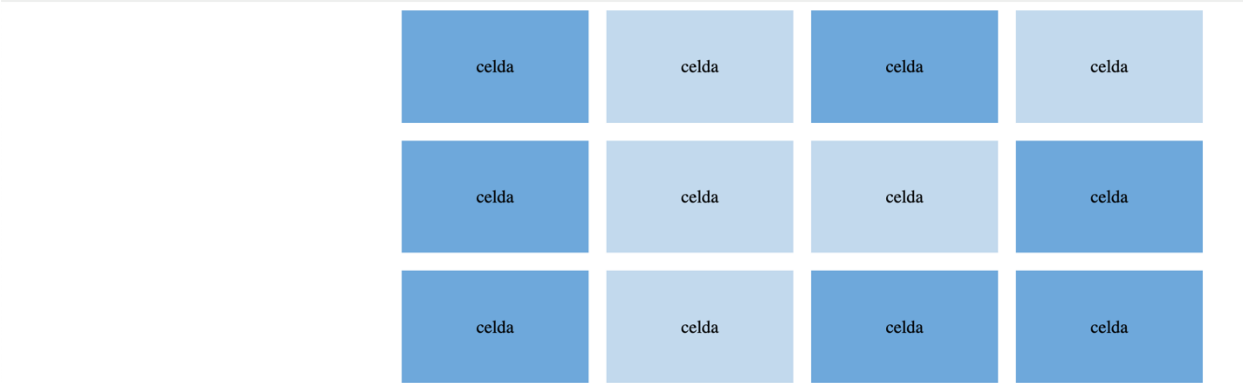
Ponte a Prueba 2 – Delegación de eventos

Se ha implementado la funcionalidad de activar o desactivar celdas en una rejilla:

- Todas las celdas dentro de un contenedor (#rejilla).
- Se ha añadido un único manejador de eventos al contenedor, en lugar de a cada celda individual.
- Al hacer click sobre una celda, se conmuta la clase activa mediante `classList.toggle()`.
- Se ha verificado que el evento no afecta al contenedor si se pulsa fuera de las celdas.

Este ejercicio ha reforzado el concepto de **delegación de eventos**, mejorando la eficiencia y el número de manejadores necesarios.

Ejemplo de ejecución:



Autor: Mario Segura Abad

Fecha de realización: 16/11/2025

Cómo visualizarlo

Requisitos:

- **Editor de código:** en mi caso, Visual Studio Code.
- **Navegador web actualizado:** en mi caso, Google Chrome.

Pasos:

1. Guardar los archivos .html y .js en la misma carpeta.
2. Abrir el archivo .html en el navegador.
3. En el **Caso Práctico 2**, pulsar los botones **Ver respuesta** para comprobar que se muestran los chistes.
4. En el **Ponte a Prueba 2**, hacer click sobre las celdas de la rejilla para activar o desactivar su estado.
5. Revisar la consola del navegador (F12 → Consola) para verificar mensajes de prueba de autoría.

Objetivos de la práctica

- Probar y comprender el código teórica del apartado 2.
- Crear manejadores de eventos asociados a elementos dinámicos.
- Implementar delegación de eventos para optimizar la gestión de interacciones.
- Documentar correctamente, **como siempre**, cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en formato PDF con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica nos ha permitido aprender y consolidar los conceptos de **eventos en JavaScript**, tanto en su definición básica como en su gestión avanzada mediante delegación.

Se ha comprobado cómo crear elementos dinámicos con eventos asociados y cómo optimizar la interacción en estructuras repetitivas como rejillas.

La práctica nos ha sido muy útil para entender cómo mejorar la eficiencia y la claridad del código en proyectos web interactivos.